Question 7: Configure continuous deployment using Kubernetes by integrating the deployment with the CI/CD pipeline and automating manifest application.

The Jenkins server from Lab 1 was extended with a continuous deployment pipeline: every push to main builds a new image, pushes it to Docker Hub, applies the manifests and updates the cluster - ending with an automated health check.

Jenkinsfile

```
// Continuous deployment pipeline - Lab 2
pipeline {
    agent any

    environment {
        HUB_IMAGE = 'abhinaaav/devops-lab2-app'
        VERSION = "2.1.${BUILD_NUMBER}"
    }

    stages {
        stage('Checkout') { steps { checkout scm } }
        stage('Build image') {
            steps {
                sh "docker build --build-arg APP_VERSION=$VERSION -f docker/Dockerfile -t $HUB_IMAGE:$VERSIO
            }
        }
        stage('Push to Docker Hub') {
            steps {
                withCredentials([usernamePassword(credentialsId: 'dockerhub',
                        usernameVariable: 'DH_USER', passwordVariable: 'DH_TOKEN')]) {
                    sh 'echo $DH_TOKEN | docker login -u $DH_USER --password-stdin'
                    sh "docker push $HUB_IMAGE:$VERSION"
                }
            }
        }
        stage('Deploy to Kubernetes') {
            steps {
                sh 'kubectl apply -f k8s/'
                sh "kubectl set image deployment/lab2-app lab2-app=$HUB_IMAGE:$VERSION"
                sh 'kubectl rollout status deployment/lab2-app --timeout=180s'
                sh 'NODE_IP=$(kubectl get nodes -o jsonpath=\
"{.items[0].status.addresses[0].address}") && curl -s http://$NODE_IP:30080/health'
            }
        }
    }
    post {
        success { echo "CD finished: SUCCESS - $HUB_IMAGE:$VERSION is live on the cluster" }
        failure { echo 'CD finished: FAILED' }
    }
}
```

Jenkins integration

- docker group membership for the jenkins user (docker builds/pushes).
- kubeconfig copied to /var/lib/jenkins/.kube with minikube cert paths rewritten to /var/lib/jenkins/.minikube-certs/ (kubectl apply, set image, rollout status).
- Docker Hub token stored as a Jenkins credential (id: dockerhub), never in the repo.
- Job devops-lab2-cd: Pipeline script from SCM, Poll SCM H/5 * * * *.

Build 1 (real failure and fix)

The first run failed only at the final health check: `minikube ip` is not available to the jenkins user (the minikube profile lives in the admin user's home). Build, push and deploy had

already succeeded. Fixed by reading the node IP through kubectl - commit `fix CD health check: use kubectl node IP instead of minikube ip`.

```
abhinav@LAPTOP-BSV7PCJK: docs/commands  —  terminal
Configure a credential helper to remove this warning. See
https://docs.docker.com/go/credential-store/
Login Succeeded
[Pipeline] sh
+ docker push abhinaaav/devops-lab2-app:2.1.1
The push refers to repository [docker.io/abhinaaav/devops-lab2-app]
44136fa355b3: Waiting
6fc242d221a6: Waiting
3eb771151506: Waiting
35f9ed196cd0: Waiting
35f9ed196cd0: Waiting
44136fa355b3: Waiting
6fc242d221a6: Waiting
3eb771151506: Waiting
44136fa355b3: Waiting
6fc242d221a6: Waiting
3eb771151506: Waiting
35f9ed196cd0: Waiting
6fc242d221a6: Waiting
3eb771151506: Waiting
35f9ed196cd0: Waiting
44136fa355b3: Waiting
35f9ed196cd0: Waiting
44136fa355b3: Waiting
6fc242d221a6: Waiting
3eb771151506: Waiting
3eb771151506: Waiting
35f9ed196cd0: Waiting
44136fa355b3: Waiting
6fc242d221a6: Waiting
6fc242d221a6: Waiting
3eb771151506: Waiting
35f9ed196cd0: Waiting
44136fa355b3: Waiting
3eb771151506: Waiting
35f9ed196cd0: Waiting
44136fa355b3: Waiting
6fc242d221a6: Waiting
```

Fig: Build 1 console: deploy succeeded, health check failed

Build 2 (SUCCESS) and the automatic build 3

```
Build #2 (manual):
+ docker build --build-arg APP_VERSION=2.1.2 ...
+ docker push abhinaaav/devops-lab2-app:2.1.2
+ kubectl set image deployment/lab2-app lab2-app=abhinaaav/devops-lab2-app:2.1.2
deployment "lab2-app" successfully rolled out
{"status":"UP","version":"2.1.2"}
CD finished: SUCCESS - abhinaaav/devops-lab2-app:2.1.2 is live on the cluster
```

```
Build #3 (auto-triggered by Poll SCM after a push):
$ kubectl get pods -l app=lab2-app -o jsonpath image field
lab2-app-5677b546d5-8zkn9: abhinaaav/devops-lab2-app:2.1.3
lab2-app-5677b546d5-mbhq4: abhinaaav/devops-lab2-app:2.1.3

$ curl health:
{"status":"UP","version":"2.1.3"}
```

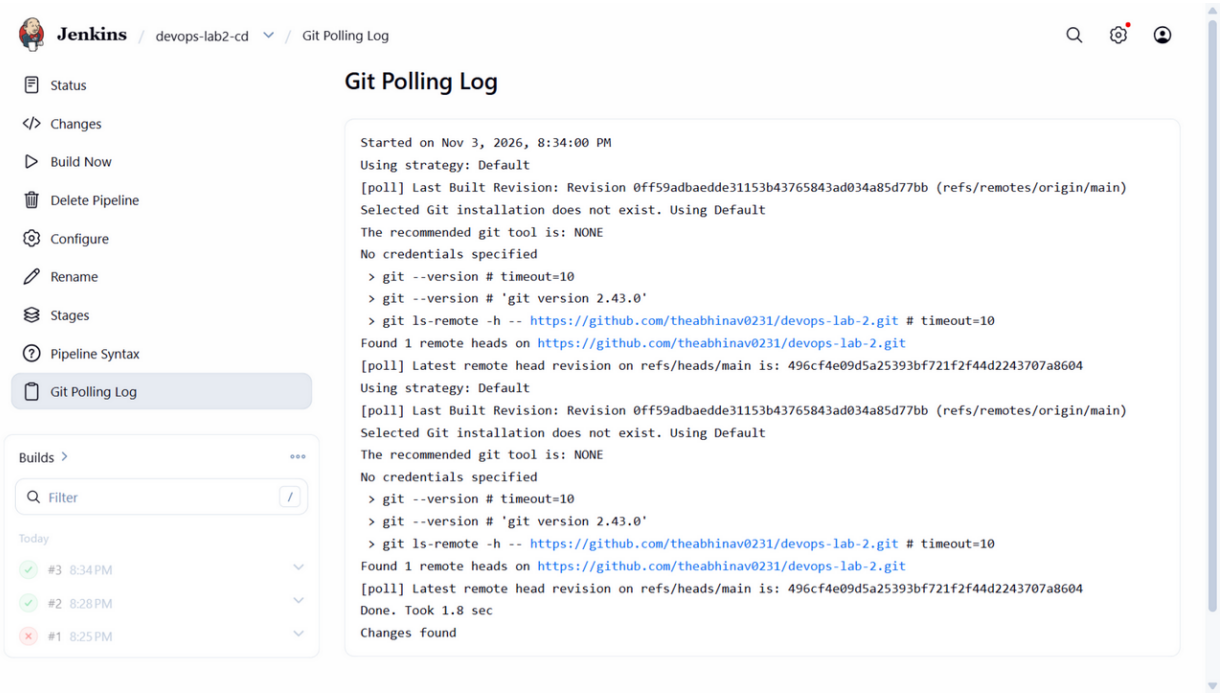


Fig: Git polling log: poll detected the push (Changes found)

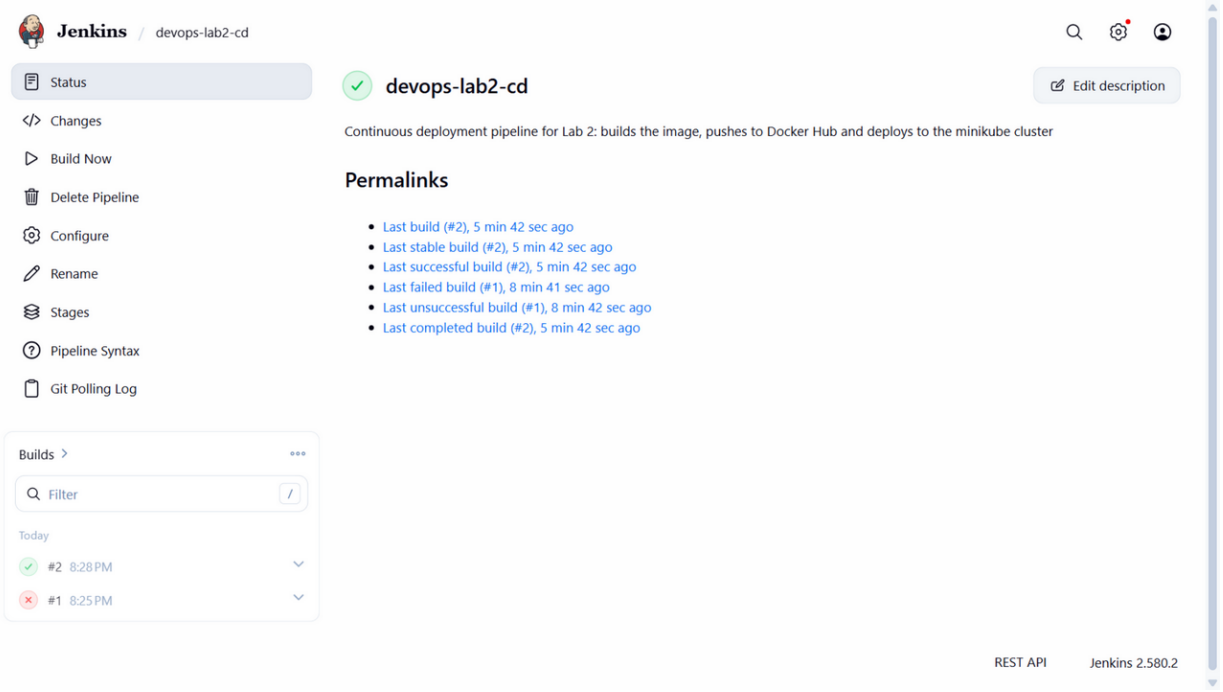


Fig: CD job page: build 1 failed, builds 2 and 3 green



Fig: Build 2 detail - commit, revision and result

Observations

- The pipeline tags images with 2.1. so every build is uniquely versioned and traceable.
- kubectl apply inside the pipeline automates manifest application; kubectl set image performs the rolling update; rollout status gates success.
- The health check at the end of the pipeline turns a bad deployment into a red build.
- A push to main reached the running cluster within one poll interval plus ~1 minute of build time.

Conclusion

All experiments were performed on Ubuntu 24.04 LTS with Docker, Docker Compose, Minikube and kubectl. The evidence (command logs, screenshots and commit history) is committed in the GitHub repository github.com/theabhinav0231/devops-lab-2 under the docs/ directory.